

```

/*
 * Experiment 6: Addressing Modes in C using GDB
 * =====
 * This program demonstrates the following addressing modes
 * as observed through GDB's disassemble and register inspection:
 *
 * 1. Immediate Addressing - Constant values in instructions
 * 2. Register Addressing - Operands held in CPU registers
 * 3. Direct Addressing - Global variable access (fixed address)
 * 4. Indirect Addressing - Access via pointer (*ptr)
 * 5. Base + Offset - Struct member access
 * 6. Indexed Addressing - Array element access (arr[i])
 * 7. Stack (Implicit) - Function call arguments via stack
 *
 * Compile:
 * gcc -g -O0 exp6_addressing_modes.c -o exp6_addressing_modes
 *
 * Run with GDB:
 * gdb ./exp6_addressing_modes
 *
 * Key GDB commands for this experiment:
 * (gdb) disassemble <function_name> -> See assembly for a function
 * (gdb) stepi -> Step one assembly instruction
 * (gdb) info registers -> See all register values
 * (gdb) x/10x $rsp -> Examine 10 words from stack pointer
 * (gdb) print &<variable> -> Print address of a variable
 */

```

```

#include <stdio.h>

```

```

/* -----
 * Global variable (Direct Addressing)
 * Lives at a fixed memory address — the compiler embeds
 * its absolute address directly in the instruction.
 * In GDB: disassemble demo_direct -> look for MOV with an
 * absolute address operand.
 * ----- */

```

```

int global_counter = 100;

```

```

/* -----
 * Struct for Base + Offset Addressing
 * Accessing a struct member compiles to:
 * MOV EAX, [base_address + offset]
 * where offset is the member's byte position in the struct.

```

```

* ----- */
typedef struct {
    int id;      /* offset 0 bytes */
    int score;   /* offset 4 bytes */
    int grade;   /* offset 8 bytes */
} Student;

/* -----
* Demo 1: Immediate Addressing
* The constant value (42) is embedded directly inside the
* machine instruction itself — no memory lookup needed.
* Assembly: MOV EAX, 42   (42 is the "immediate" value)
* ----- */
void demo_immediate() {
    int a = 42;      /* Immediate: 42 is baked into the instruction */
    int b = 10;      /* Immediate: 10 is baked into the instruction */
    int c = a + b;
    printf("[Immediate] a=%d, b=%d, a+b=%d\n", a, b, c);
}

/* -----
* Demo 2: Register Addressing
* CPU operates on values already loaded into registers.
* The compiler keeps frequently used variables in registers
* (EAX, EBX, etc.) to avoid slow memory accesses.
* Assembly: ADD EAX, EBX  (both operands are registers)
* ----- */
void demo_register() {
    register int x = 5; /* Hint to compiler: keep in register */
    register int y = 3;
    int result = x + y; /* ADD instruction: register + register */
    printf("[Register] x=%d, y=%d, x+y=%d\n", x, y, result);
}

/* -----
* Demo 3: Direct (Absolute) Addressing
* Accessing a global variable uses its fixed memory address.
* Assembly: MOV EAX, [0x601030] (absolute address of global_counter)
* ----- */
void demo_direct() {
    global_counter += 10; /* Access via fixed address */
    printf("[Direct] global_counter = %d (address: %p)\n",
        global_counter, (void*)&global_counter);
}

```

```

/* -----
* Demo 4: Indirect (Pointer) Addressing
* The address of the target is held in a pointer variable.
* The CPU first reads the pointer to get the address, then
* fetches the value at that address.
* Assembly: MOV EAX, [EBX] (EBX holds the address)
* ----- */
void demo_indirect() {
    int value = 77;
    int *ptr = &value;    /* ptr holds the address of value */
    int fetched = *ptr;    /* Dereference: go to address in ptr */
    printf("[Indirect] value=%d, ptr=%p, *ptr=%d\n",
        value, (void*)ptr, fetched);

    /* Pointer arithmetic: move to next int */
    int arr[3] = {10, 20, 30};
    int *p = arr;          /* p points to arr[0] */
    printf("[Indirect] *p=%d, *(p+1)=%d, *(p+2)=%d\n",
        *p, *(p+1), *(p+2));
}

/* -----
* Demo 5: Base + Offset Addressing
* Struct member access uses the struct's base address plus
* a compile-time-known offset for each field.
* Assembly: MOV EAX, [EBX + 4] (EBX = &student, 4 = offset of score)
* In GDB: disassemble demo_base_offset -> note the constant offsets
* ----- */
void demo_base_offset() {
    Student s;
    s.id = 1;    /* [base + 0] */
    s.score = 95; /* [base + 4] */
    s.grade = 1; /* [base + 8] */

    printf("[Base+Offset] id=%d (offset 0), score=%d (offset 4), grade=%d (offset 8)\n",
        s.id, s.score, s.grade);
    printf("[Base+Offset] base address of struct: %p\n", (void*)&s);
}

/* -----
* Demo 6: Indexed Addressing
* Array element access uses base address + index * element_size.
* Assembly: MOV EAX, [EBX + ECX*4] (ECX = index, 4 = sizeof(int))

```

```

* ----- */
void demo_indexed() {
    int marks[] = {85, 90, 78, 92, 88};
    int n = 5;

    printf("[Indexed] Array base address: %p\n", (void*)marks);
    for (int i = 0; i < n; i++) {
        /* marks[i] compiles to: base_address + i * sizeof(int) */
        printf("[Indexed] marks[%d] = %d (address: %p)\n",
            i, marks[i], (void*)&marks[i]);
    }
}

/* -----
* Demo 7: Stack (Implicit) Addressing
* Function arguments and local variables are pushed onto
* the stack. The CPU uses ESP/RSP (stack pointer) with
* implicit offsets to access them.
* In GDB: x/10x $rsp -> examine the stack when inside this function
* ----- */
int stack_demo_helper(int p, int q, int r) {
    /* p, q, r are on the stack (or in registers per calling convention) */
    /* In GDB: 'info args' shows these; 'x $rsp' shows the raw stack */
    printf("[Stack] Inside helper: p=%d (addr %p), q=%d (addr %p), r=%d (addr %p)\n",
        p, (void*)&p, q, (void*)&q, r, (void*)&r);
    return p + q + r;
}

void demo_stack() {
    int a = 5, b = 10, c = 15;
    int total = stack_demo_helper(a, b, c); /* args pushed to stack / passed in registers */
    printf("[Stack] Total = %d\n", total);
}

/* -----
* main: Calls all demos in sequence
* Set a breakpoint at each demo call to step through them.
* ----- */
int main() {
    printf("=== Experiment 6: Addressing Modes via GDB ===\n\n");

    demo_immediate();    printf("\n");
    demo_register();     printf("\n");
    demo_direct();       printf("\n");

```

```

demo_indirect();    printf("\n");
demo_base_offset(); printf("\n");
demo_indexed();     printf("\n");
demo_stack();       printf("\n");

printf("=== All addressing modes demonstrated ===\n");
return 0;
}

```

OUTPUT:

```

=== Experiment 6: Addressing Modes via GDB ===

[Immediate] a=42, b=10, a+b=52

[Register] x=5, y=3, x+y=8

[Direct] global_counter = 110 (address: 00404004)

[Indirect] value=77, ptr=0061FEF0, *ptr=77
[Indirect] *p=10, *(p+1)=20, *(p+2)=30

[Base+Offset] id=1 (offset 0), score=95 (offset 4), grade=1 (offset 8)
[Base+Offset] base address of struct: 0061FEF4

[Indexed] Array base address: 0061FEE4
[Indexed] marks[0] = 85 (address: 0061FEE4)
[Indexed] marks[1] = 90 (address: 0061FEE8)
[Indexed] marks[2] = 78 (address: 0061FEEC)
[Indexed] marks[3] = 92 (address: 0061FEF0)
[Indexed] marks[4] = 88 (address: 0061FEF4)

[Stack] Inside helper: p=5 (addr 0061FEE0), q=10 (addr 0061FEE4), r=15 (addr 0061FEE8)
[Stack] Total = 30

=== All addressing modes demonstrated ===

```